

RMOEA/D 求解双目标模糊柔性作业车间调度

——复现、评价口径纠正与组件有效性再评估

2026 年 9 月 21 日

摘要

一批工件要在多台机器上依次加工、每道工序可以任选机器、实际耗时事先说不准，怎样安排才能既缩短总工期又均衡机器负载？这类**双目标模糊柔性作业车间调度问题**是典型的 NP 难组合优化问题。Li 等 (2022) 提出的 RMOEA/D 用 Q-learning 同时驱动参数自适应 (Q-PAS) 与变邻域算子选择 (RVNS)，在该问题上报告了良好效果。本文在 Brandimarte Mk01–Mk10 上对该算法做了可复现的重实现，并在统一评价口径下重新审计其每个组件。本文的工作有三点。

其一，评价口径。文献与复现中普遍使用的一种归一化取法——按参与比较的前沿确定边界（下称盒口径）——有**两个结构性缺陷**：边界随实例集与臂集漂移，且系统性放大效应量。实测同一批数据下按臂对汇总的放大为 $4.9\times$ 至 $29.7\times$ 、逐格子最高 $47.9\times$ ，**倍数并非常数**；更严重的是它会改变现象的形状：把“少数实例显著、其余与 0 不可区分”抹成“处处微小为正”。本文因此主张改用由实例数据确定性推出的**实例边界口径**，并给出可复现的对照证据。

其二，组件有效性的重新判定。在实例边界口径下，六个阶梯臂的逐级增量为 MIX3 初始化 $+3.406\%$ (10/10 实例胜出)、随机 VNS $+0.421\%$ 、精英档案 $+0.082\%$ ，而 Q-PAS 仅 $+0.030\%$ ($p = 0.625$ ，不显著)、RL 选算子 -0.003% 。进一步用 anytime 轨迹证明算力在 $G = 200$ 附近已接近饱和，因此 RL 组件的失效不是“学不到”，而是**没有空间可学**。

其三，可事前判定的增益。我们提出一个零代探针：只调用一次初始种群构造（不做任何搜索），用初始前沿 HV 的杠杆 λ_0 预测“某个新实例上 MWR 初始化相对论文口径是否真有净增益”，并在 Hurink e-data 的**全部 66 个实例**上做完全预注册的留出验证。**该门控没有通过这次检验**：它在开发集上与净增益完全分离，在留出集上的判对率却低于“一律拒绝”这条平凡基线，放行实例的精确率仅 0.19。我们据此把“廉价零代门控可以跨分布移植”这个常见默认写成明确的负结果。

关键词：柔性作业车间调度；分解型多目标进化算法；模糊加工时间；强化学习；消融实验；评价口径；可复现性

1 引言

问题从哪里来。 工厂排产有个日常难题：一批工件要依次经过多道工序，每道工序占用一台机器，怎么安排才能既让整批活尽早干完、又不让少数机器长期超载？若每道工序可以在多台机器中任选（称**柔性**），可选方案数随规模指数增长；若各工序的实际耗时事先说不准（称**模糊**），问题更难。这类**双目标模糊柔性作业车间调度问题** (bi-objective fuzzy flexible job-shop scheduling

problem, FFJSP) 是典型的 NP 难组合优化问题, 要在“工期”与“负载均衡”两个互相冲突的目标之间做取舍 (这些概念在 §2 有面向非专业读者的说明, 不熟悉的读者可先读那一节)。

近年的主流做法是把**进化算法**与**强化学习**结合: 让算法在搜索过程中“学会”自己的参数该取多少、该用哪种局部搜索算子。Li 等 [1] 提出的 RMOEA/D 即属此类——它把 Q-learning 同时接在两个决策点上: 一是每代选择 Tchebycheff 分解中的参数 T (该文称 Q-PAS), 二是选择变邻域搜索的算子 (RVNS), 并在 Brandimarte 基准上报告了相对基线的改进。

复现中的两个隐形陷阱。 复现这类“学习型元启发式”时, 有两个容易出错、且出错方式很隐蔽的地方——它们不会报错, 只会悄悄改变结论:

(1) **多目标指标的评价口径。** 超体积 (HV) 需要把两个目标归一化到同一量纲。把归一化边界取为“参与比较的前沿的极值” (盒口径) 是一行代码就能写出的做法, 但它使边界成为一个**依赖于臂集与实例集**的量: 同一份数据, 只要换掉比较对象, HV 读数就变。这在复现里意味着一个组件“有没有效果”的结论会随同批实验的设计而漂移。

(2) **比较基准与问题的不对齐。** 当要回答“该不该用我们提出的初始化”时, 正确的对照是**论文自己的初始化**; 而“随机初始化”只能回答“有个好起点值多少”。这两个量在同一批数据上都算得出来, 但用前者去论证后者是无效的。

本文主要发现 (一段话)。 在统一、可复现的评价口径下, 本基准与算力预算之内真正有效的是**初始化** (即搜索起点) 与**算力总量** (且后者很快饱和); 两个 RL 组件没有可实现空间——这并非“学习算法不行”, 而是它在**两个方向上都没有事可做**。在建设性的一面, 我们提出一个廉价的**零代探针**, 用于事前判定“某个新实例上初始化的净增益会不会出现”, 并在全部 66 个独立实例上做完全预注册的检验——**它没有通过**: 跨到留出集之后, 它的判别力低于“一律拒绝”这条平凡基线。本文把这条负结果完整报告出来, 它正是本文方法学主张的自我检验: 门限若只在挑出它的那批实例上成立, 就不能算门控。

阅读地图。 只关心结论的读者读本节与 §6、§7、§10 即可; 关心方法学的读者重点看 §4 (评价口径) 与 §5.3 (统计口径); 想复现的读者请从 §5 与随文脚本入手。

贡献。

- 量化了盒口径的偏差: 同一批数据、同一臂对, 效应量被放大约一个数量级, 且**放大倍数不是常数** (按臂对汇总 $4.9\times-29.7\times$, 逐格子最高 $47.9\times$), 显著性检验的 p 与胜出场数同样改变 (§4)。
- 在实例边界口径下重新给出六臂消融阶梯的逐级效应, 并把“算力饱和”量化, 由此把 RL 组件的失效解释为**动作空间与算力上都没有空间** (§6)。
- 提出零代探针门控 ($\lambda_0 =$ 初始前沿 HV 杠杆): 它在开发集上与净增益完全分离, 但在 66 个实例的预注册留出集上**没有通过**——判对率 0.70 低于“一律拒绝”的平凡基线 0.85, 放行实例的精确率仅 0.19, 召回仅 0.30 (§7); 此外回答“门控需要多少 seed” (§8)。
- 全部脚本、原始日志与“一个数字一个来源”的导出流程随文公开, 论文中每个数字都由 `scripts/paper_export.py` 从 `logs/` 现场生成。

2 预备知识：读懂本文所需的五个概念

本文的方法学结论不依赖多目标优化的专门知识，但需要先建立下面几个概念。每小节末尾用一句话说明“在本文中”它对应什么；已经熟悉这些内容的读者可只读那一句，再跳到 §4。表 1 是全文记号速查。

2.1 车间调度：在不确定中把活排完

一个车间有若干**工件**（待加工的产品）和若干台**机器**。每个工件要依次经过若干道**工序**，每道工序占用一台机器、消耗一段时间。调度要决定两件事：**每道工序派给哪台机器、同一台机器上谁先谁后**。若每道工序只能由一台指定机器完成，这类问题叫作业车间调度（JSP）；若每道工序可以在多台机器的任一台上完成，就叫**柔性**作业车间调度（FJSP）——“柔性”留下了取舍空间，也正是算法要去搜索的东西。

现实中加工时间很少提前精确已知。本文沿用复现对象 [1] 的做法，把每道工序的加工时间写成一个**三角模糊数** $\tilde{t} = (t_1, t_2, t_3)$ ： t_1 为乐观值、 t_2 为最可能值、 t_3 为悲观值。模糊数之间的比较与折算规则同样沿用该文约定 (§3)。由于模糊区间由实例与随机种子共同决定，同一实例在不同种子下得到的时间并不相同——这一点直接决定了本文的统计做法 (§5.3)。

在本文中：这句话解释了两个纪律的来源——为什么所有比较都要**同种子配对**，以及为什么结论必须在多个实例上重复才可信。

2.2 两个互相冲突的目标：没有“唯一最优解”

本文同时追求两件事：**整批活尽早干完**（最大完工时间 $C_{\max}$ ，最后一道工序的结束时刻）与**机器负载不要过度集中**（机器总负载 W ，各机器负荷之和）。这两个目标会打架：想缩短 $C_{\max}$ ，常要把工序塞给空闲机器，总负载反而上升；想压低总负载，又可能把工期拖长。因此**不存在“两个目标都最好”的单一排产方案**。

比较两个方案的办法叫 **Pareto 支配**：若方案 A 的两个目标都不比 B 差、且至少有一个更好，就说“A 支配 B”，B 可被淘汰。把所有互不支配的方案挑出来，得到 **Pareto 前沿**（图 1a）。它不是一个点，而是一条**取舍曲线**：曲线上的每个方案都是某个侧重下的最优。

在本文中：算法的输出不是“一个排产方案”，而是这样一条前沿；因而“算法好不好”也必须用前沿之间的比较来回答，这是 HV 存在的理由。

2.3 用一个数评价一条前沿：超体积 HV

要比较两个算法，就需要把“一条前沿”压成一个数。常用的量是**超体积** (hypervolume, **HV**)：先选定一个比所有方案都差的**参考点** ref，HV 取“前沿与 ref 之间围出的面积”（图 1b）——前沿越靠近原点（两个目标都更小），面积越大，算法越好。

但 $C_{\max}$ 与 W 的**量纲与量级都不同**（一个是时间，一个是若干机器时间之和），直接算面积没有意义，必须先把两个目标**归一化**到同一尺度。归一化需要为每个目标指定**下界 lo** 与**上界 hi**。**这组边界怎么取，正是本文的核心方法学议题** (§4)：一种常见取法只在当批实验内部成立，换一批比较对象读数就变。

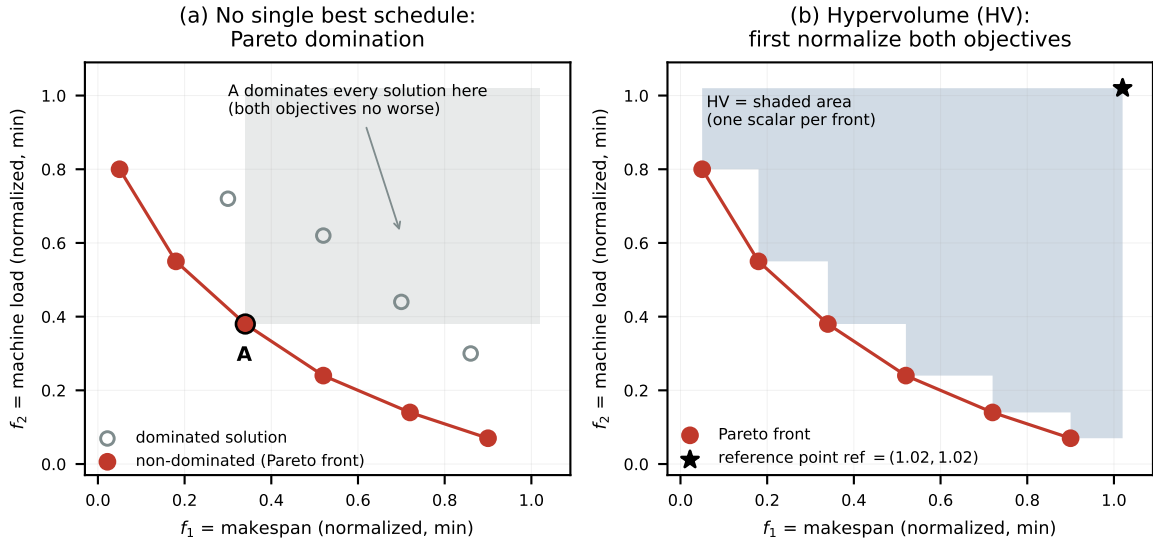


图 1: 概念示意图——点 A 为手工构造，非实验结果，不参与本文任何结论。(a) Pareto 支配：解 A 的两个目标都不差于阴影区内任何解，故那些解“被 A 支配”；互不支配的解（红）连成 Pareto 前沿。(b) 超体积 HV：双目标要先归一化（两轴各自缩放到 $[lo, hi]$ ），HV 才等于前沿与参考点之间围出的面积；而 lo/hi 如何取定正是 §4 的主题。

在本文中：正文一律采用“由实例数据确定性推出、与比较对象无关”的边界（实例边界口径），并在 §4 量化另一种取法（盒口径）的后果。

2.4 MOEA/D：把多目标问题拆成许多单目标问题

Pareto 前沿上有无穷多个取舍点，想一次全找到很难。**分解型**算法把它拆开：预先给定一组**权重向量**（例如“七成看工期、三成看负载”），每个权重对应一个加权单目标子问题；子问题各自求解、并与邻居交换信息，合起来铺满整条前沿。把多个目标合成一个标量的公式有多种，本文复现对象采用的是 Tchebycheff 形式。MOEA/D[3] 是这一类的代表。另一条主流路线不做分解，而是按“谁支配谁”给解排序、并保留整层互不支配的解，代表是 NSGA-II[2]；本文不比较这两条路线，提到它只是为说明 MOEA/D 在算法谱系中的位置。

权重向量的集中程度由一个参数控制，记为 T ： T 小则各子问题关注点差异大、前沿铺得开； T 大则铺得均匀但彼此接近。本文复现的 RMOEA/D[1] 用 **Q-learning**（一种“记下每次选择的好坏、下次更倾向选好的”的强化学习方法）在**两个决策点**上做选择：(i) 每代该用哪个 T （该文档称 **Q-PAS**）；(ii) 该用哪种变邻域搜索算子来改进解（该文档称 **RVNS**）。此外算法还有一个**精英档案**，用于保存搜索过程中见过的最好前沿。

在本文中：上述每个“决策点”都是一个可开可关的**组件**；本文把它们排成一条阶梯、逐级打开，检查每一级究竟有没有贡献 (§6)。正文把阶梯上的一个配置称为一个**臂** (arm)——这个词借自老虎机问题：把每个组件看成一台老虎机，问“拉它到底值不值”。相应地，逐个开关组件、观察各自贡献的实验叫**消融实验** (ablation)。

表 1: 全文记号与术语速查（按首次出现的顺序）。

记号 / 术语	含义
FJSP / FFJSP	柔性作业车间调度 / 其双目标模糊版本；本文的问题对象
$C_{\max}, W$	两个目标：最大完工时间、机器总负载；都越小越好
Pareto 支配 / 前沿	“两项都不差且至少一项更好”；互不支配的解构成的取舍曲线
HV	超体积：前沿与参考点 ref 围出的面积，把一条前沿压成一个数
ref	HV 的参考点，本文取 (1.02, 1.02)（归一化后）
实例边界口径	HV 归一化边界由实例数据确定性推出， 与比较对象无关 （本文采用）
盒口径	边界取“参与比较的前沿的极值”，读数随比较对象漂移（本文仅作对照）
臂 (arm)	阶梯上的一个算法配置；六个臂 D_1 – D_5 与 RMOEAD
RMOEAD	完整算法 (RL 选算子)；与 D_5 仅差“谁选算子”
Q-PAS / RVNS	论文的两个 RL 组件：选参数 T / 选邻域搜索算子
MIX3 / MWR	两种初始化（搜索起点）：MIX3 是论文自带的（三等分：随机 / 最短时间 / 负载最轻，工序顺序均随机）；MWR 是本文要检验的候选，额外固定了工序顺序
目标 A / 目标 B	分别以随机初始化 / 论文自身初始化 (MIX3) 为对照的净增量
λ_0 (零代探针)	只构造一次初始种群（不做任何搜索）得到的初始前沿 HV 杠杆
实例级 / run 级	先在实例内对 30 个种子取均值 ($n = \text{实例数}$) / 逐种子配对 ($n = 30 \times \text{实例数}$)
d_z / p	配对效应量 / 配对 Wilcoxon 检验的显著性水平
等算力比较	比较两个配置时给它们相同的算力预算，否则“更好”可能只是“算得更多”
算力响应面	把算力当自变量、观察效应如何随之变化的一组实验
墙钟预算	按真实流逝的时间（而非代数）给算力分档；同代数不等于同耗时
置换零假设	打乱实例与目标量的对应、重复多次。用于家族校正：同时考察多个候选并从中挑最大时，“挑最大”本身就会带来偏差，置换检验正是模拟这个动作

2.5 本文用到的四个统计工具

- **配对比较**：让两个配置在“同一实例、同一随机种子”下各跑一遍，比较它们的差，而不是比较各自的绝对值。共同因素（实例难度、本次抽到的模糊时间）在相减时被消掉，剩下的差才可归因于被比较的组件。本文的每一组比较都是配对的。
- **Wilcoxon 符号秩检验**：配对差往往不服从正态分布，用不了 t 检验，于是改用只看“差的正负与其排名”的非参数检验，得到 p 值。本文把“某组件有增益”严格定义为“**配对检验** $p < 0.05$ 且方向为正”，而不是“差不是零”——浮点误差会让任何差都不等于零，用“非零”当判据会把所有实例都判成有增益 (§5.3)。
- **Cohen’s d_z** ：效应量，衡量“差相对于其自身波动有多大”。 p 值只回答“有没有”， d_z 回答“有多大”，二者必须同时报告。
- **置换检验**：当同时考察很多候选量、又从中挑最大者时，普通的显著性门槛会失效（“挑最大”这个动作本身就带来偏差）。做法是把实例与目标量的对应关系随机打乱很多次，看“乱着也能得到这么大”有多常见。本文用它做多重比较的家族校正，也用它检验“某实例还有提升空间”这类事后论断是不是噪声 (§5.3)。

3 问题、算法与指标

3.1 双目标模糊 FJSP

给定 n 个工件、 m 台机器，每个工件的每道工序的加工时间用三角模糊数 $\tilde{t} = (t_1, t_2, t_3)$ 表示，且可在其可行机器集合中任选一台加工。目标是

$$\min F = (f_1 = C_{\max}, f_2 = W), \quad (1)$$

其中 $C_{\max}$ 为最大完工时间， W 为机器总负载。三角模糊数的比较与“模糊 $\rightarrow$ 确定” (crisp, 即把模糊数折算成一个可直接比较的确定值) 的规则沿用 Li 等 [1] 的约定，以保证与本项目复现对象一致；模糊区间由实例名与随机种子共同决定，因此同一实例在不同 seed 下的模糊时间不同，这使“同 seed 配对”成为一切组间比较的基本单位。

3.2 超体积 (HV)

本文的单一汇总指标是前沿的 HV (几何含义见 §2)，参考点固定为 $\text{ref} = (1.02, 1.02)$ 。HV 的归一化边界取法有两种，本文的全部结论都建立在第二种之上 (§4)：

- **实例边界口径** (本文采用)：边界 (lo, hi) 由实例数据确定性推出 (下界取临界路径下界、上界取全部工序最长时间之和)，与参与比较的臂集无关。实现见 `instance_hv_bounds()`，算法落盘的 `final_hv` 用的就是它。
- **盒口径** (仅作对照)：边界取参与比较的前沿的极值，实现见 `estimate_hv_bounds()`。

3.3 RMOEA/D 的可插拔组件

复现中我们把算法拆成六个臂 (表 3 的“阶梯”列)，每一级相对上一级只打开一个组件： D_1 为纯 MOEA/D (固定 $T = 10$ ，关闭 MIX3/RVNS/Q-PAS/精英档案)； D_2 打开 MIX3 初始化； D_3 打开随机 VNS； D_4 把固定的 T 换成 Q-PAS； D_5 打开精英档案；RMOEAD 把随机 VNS 换成 RL 选算子。这一阶梯与文献 [1] 表 5 的步骤分解对齐，因而“每一级是否有效”就是对该文主张的逐条检验。RMOEAD 与 D_5 只差“谁选算子”，是检验 RL 组件**最干净的单变量对照** (等算力、同其它一切)。

4 评价口径：一个会改变结论的隐藏变量

本节是全篇的方法学核心，用一句话概括它的主张：同一份实验结果，只要换一种“把多目标压成单一指标”的做法，就能读出不同的效应量、不同的显著性、甚至不同形状的结论。图 2 用同一批前沿点配两种边界把这件事讲清楚：

下面先说明两种边界取法的区别 (§4.1)，再量化后果 (§4.2)，最后给出本文遵守的报数规则。

4.1 盒口径的两个缺陷

(a) **跨实例污染**。若把多个实例的前沿并进同一个盒，这个盒就不得不同时容纳规模相差悬殊的实例。先看一个不需要任何优化知识的量：**负载下界**——把该实例全部工序都取最短加工时

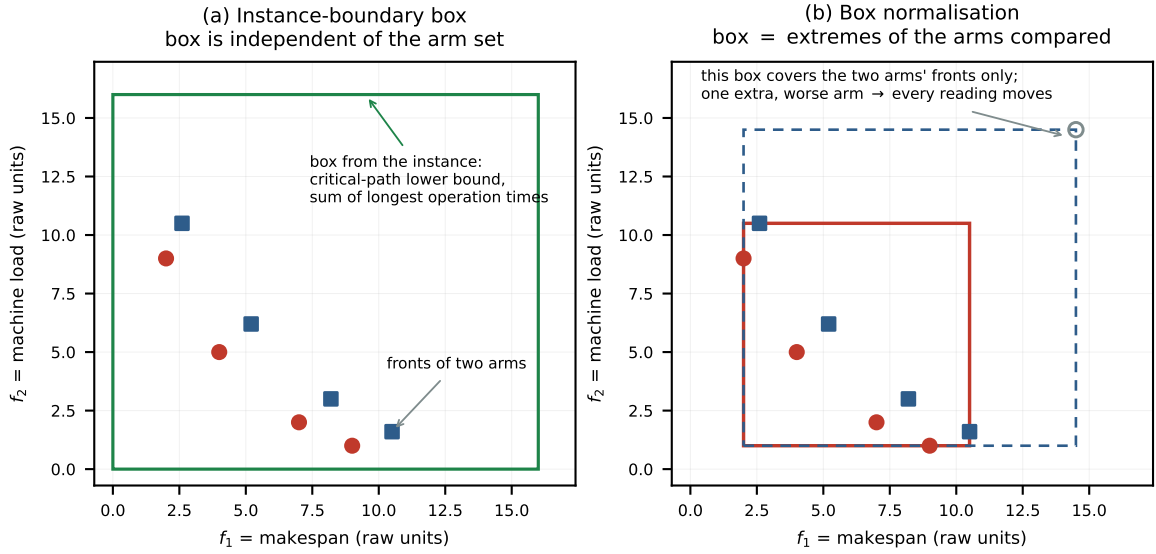


图 2: 评价口径示意图——点为手工构造，非实验结果，不参与本文任何结论。两栏用的是同一批前沿点（红圆与蓝方分别代表两个臂的前沿），唯一的差别是“灰框怎么定”。(a) **实例边界口径**：框由实例数据推出（下界取临界路径下界、上界取全部工序最长时间之和），换一批算法来比，框也不会变，因此读数可以跨实验引用。(b) **盒口径**：框取“参与比较的臂的前沿极值”，于是框恰好贴着数据——再放进一个更差的臂（灰圈），框的上界就被撑高（蓝虚线），同一批点的归一化坐标随之改变。两栏横轴均为完工时间 f_1 、纵轴均为机器总负载 f_2 ，都是未归一化的原始单位，越靠左下越好。

间再加起来，就是机器总负载 W 再优化也到不了的理论最小值。在开发集内，仅这一项，最大与最小就相差 $18.1 \times$ （Mk08 的 2453 对 Mk02 的 136），最大完工时间的下界也相差 $8.5 \times$ 。规模差这么远的前沿被塞进同一个框后，两头的读数都被“拉”进相近区间，于是读出“所有变体都近乎完美”的假象。凡按（臂, seed）在多个实例上摊平再取极值的写法都会踩这一点。

(b) **臂集污染**。盒只应包含**参与比较的臂**。若把与本次比较无关的臂一并放进盒（例如把另做的一批 T 扫描臂混进消融实验的日志），上界被无关数据撑高，同一份数据的所有读数随之改变。本文的对照实验因此**逐臂对**重算边界，即盒内恰好只有被比较的两个臂。

4.2 量化的后果

表 3 右三列给出同一批数据（8 臂 $\times$ 10 实例 $\times$ 30 seeds）在两种口径下的读数。三点值得注意：

1. **放大倍数不是常数**。如果两种口径之间只差一个固定倍数，那么把盒口径的数字除以该倍数即可折回实例边界口径，问题尚不算棘手。事实并非如此：按臂对汇总（含义是“先各自求均值、再作比”，见 §5.3），放大倍数从 $4.9 \times$ 到 $29.7 \times$ 不等，逐格子最高达 $47.9 \times$ （Mk10 的 $D_4|D_3$ ：实例边界 -0.017% $\rightarrow$ 盒口径 -0.829% ）。因此“盒口径数字除以固定倍数即可折回”是错的。

那么被放得最狠的是哪些格子？图 3 把两种口径并排画出： $D_2|D_1$ 的相对增幅被放大 $5.9 \times$ ，而几乎为零的 $D_4|D_3$ 被放大 $10.6 \times$ ——**越接近零的效应，相对放大越厉害**。在全部 50 个（实例，臂对）格子里， $|\Delta HV| \leq 0.02\%$ （“接近零”的显式阈值）的 9 个格子放大中位数为 $16.7 \times$ ，其余 41 个只有 $7.9 \times$ 。**这里的秩相关必须写明“取了对数”这一步**：秩相关的意思

是“不比数值大小、只比名次”。先把放大倍数取 $\log_{10}$ 再排名次，得 $\text{Spearman} = -0.37$ ($p = 0.009$)；若直接对原始放大倍数排名次则只有 -0.25 ($p = 0.076$ ，未过线)。两者方向一致、强弱不同——不写出这一步，读者无法复核这个数。这正是“任何数字离开其口径不可引用”的一个具体例子。

2. p 值与胜出场数也会变。这一点比幅度更容易被忽略：口径改变不只是“数字大小”，而是显著性判定本身。
3. 现象的形状会变。本文 §6.3 会遇到一个“3/10 实例显著、7/10 与 0 不可区分”的双峰形态；在盒口径下，那 7 个非显著实例变成从 -0.58% 散到 $+0.95\%$ 的一团（中位 $+0.29\%$ ，且其中 3 个翻转了符号），于是被读成“效应弱但普遍存在”。形状的失真比幅度的失真更难察觉。

4.3 报数规则

本文后续一律遵守：效应量与显著性只报实例边界口径；对照展示盒口径时必须同时给出放大倍数；任何数字离开其口径不可引用；跨实例只比相对差与秩，不比 HV 绝对值（各实例边界不同）。

5 实验设置

5.1 实例集

基准实例好比“标准考题”：同一组考题上跑不同算法，结果才可比。表 2 列出本文使用的两组。开发集为 Brandimarte Mk01–Mk10[4]——复现对象所用的原始基准，规模由小到大——用于复现实验与标定门限；留出集为 Hurink e-data 的全部 66 个实例 [5]，只在门限冻结之后启用一次，用来检验门限能否推广到新分布。这里有个容易被忽略的前提：表 2 两段的“弹性比”列（该列含义是“每道工序平均有多少台机器可选”）显示，这一族的柔性远低于开发集——每道工序平均只有一台多一点机器可选，而 Mk01–Mk10 最高到四台。所以留出验证不只是换一批实例，更是在换一个问题分布；§7 的负结果与这个差异直接相关。留出集的选择完全预注册：所谓预注册，指先写明“将使用哪批数据、按什么规则判定”，再去看结果，以排除事后挑选有利于自己的实例；本文的做法是取该数据集的全部实例，不做任何按结果的取舍。实例文件取自公开基准库并以 SHA-256 记录来源（data/hurink/PROVENANCE.json）。

5.2 参数与算力

算法本质上是随机搜索：一个随机种子（seed）决定它内部的所有随机选择，换一个种子就是一次独立重复。单次重复的结果可能碰巧偏好或偏坏，因此每个配置都要跑多个种子。本文取种群规模 $N_p = 100$ （“种群”就是算法同时维护的那一组候选解，上文的每个权重向量各占一个）、代数 $G = 200$ （迭代多少轮）、每个配置 30 个随机种子（编号 42–71）， $\text{ref} = (1.02, 1.02)$ 。所有组间比较都做同 seed 配对，以消掉“模糊时间随 seed 变”带来的实例内噪声。响应面实验 (§6.2) 另在 $G = 2000$ 上限下按墙钟预算分档。“墙钟”（wall-clock）指真实流逝的时间，它和“跑了多少代”不是一回事：同样的代数，不同配置实际花掉的时间并不相同。要比较“给更多算力有没

表 2: 实例集特征: (a) 开发集 Brandimarte Mk01–Mk10; (b) 留出集 Hurink *e*-data, 按 (工件, 机器, 工序) 分组合并。两段均续排为左右两栏

(a) Brandimarte Mk01–Mk10: 复现与消融的开发集											
实例	工件	机器	工序	弹性比	c_v	实例	工件	机器	工序	弹性比	c_v
Mk01	10	6	55	2.09	0.56	Mk06	10	10	150	3.27	0.63
Mk02	10	6	58	4.10	0.50	Mk07	20	5	100	2.83	0.64
Mk03	15	8	150	3.01	0.63	Mk08	20	10	225	1.43	0.48
Mk04	15	8	90	1.91	0.63	Mk09	20	10	240	2.52	0.43
Mk05	15	4	106	1.71	0.32	Mk10	20	15	240	2.98	0.47

(b) Hurink <i>e</i> -data: 66 个实例, 全部用作 独立留出集 (不挑选)									
实例数	工件	机器	工序	弹性比	实例数	工件	机器	工序	弹性比
×1	6	6	36	1.17	×4	20	5	100	1.13
×1	7	7	49	1.18	×2	20	5	100	1.14
×3	10	5	50	1.20	×9	10	10	100	1.14
×2	10	5	50	1.18	×2	15	10	150	1.15
×1	13	4	52	1.21	×1	15	10	150	1.14
×1	11	5	55	1.20	×2	15	10	150	1.16
×1	14	4	56	1.20	×4	20	10	200	1.14
×1	12	5	60	1.18	×1	20	10	200	1.13
×1	10	6	60	1.17	×3	15	15	225	1.15
×1	8	8	64	1.17	×2	15	15	225	1.14
×1	8	9	72	1.17	×2	30	10	300	1.14
×4	15	5	75	1.15	×3	30	10	300	1.13
×1	15	5	75	1.16	×2	20	15	300	1.13
×1	10	10	99	1.13	×1	20	15	300	1.13
×8	10	10	100	1.13					

弹性比 = 每道工序可选机器数的均值; c_v = 模糊三角加工时间的变异系数。Mk01–Mk10 用于复现、消融与**门限标定**; (a) 段 10 行续排为 5+5。Hurink *e*-data 共 66 个实例, 规模 36–300 道工序; (b) 段左右两栏续排, 按 (工件, 机器, 工序) 分组合并, × c 为组内实例数。全部实例都进留出集, **没有任何按结果的事后挑选**。

有用”, 按真实耗时来分档才公平, 所以响应面用墙钟而不用代数, 用于观察 “算力变大时效应如何变”。

5.3 统计口径

同一组比较给两套读数: **实例级** (先在实例内对 30 个 seed 取均值, $n = 10$ 或 $n = 66$) 与 **run 级** (逐 seed 配对, $n = 300$)。检验用 Wilcoxon 符号秩 (§2), 效应量用配对 Cohen’s d_z ; 两者的分工是 “ p 回答有没有、 d_z 回答有多大”, 本文两者同时报告。

相对增幅一律 “先求均值、再作比”, 即写成 $\overline{\Delta\text{HV}}/\overline{\text{HV}}$ 上一级, 而不写 $\overline{\Delta}/\overline{\text{HV}}$ ——后者在效应 ≈ 0 时, 分子分母会因正负相消而反号。盒口径的放大倍数也守同一约定: $\overline{\Delta\text{HV}}_{\text{盒}}/\overline{\Delta\text{HV}}_{\text{实例}}$, 而不是逐 run 放大倍数的平均, 两者相差可达数倍。

存在多个候选预测变量时, 一律用**置换零假设** (固定预测变量的秩矩阵、只打乱实例与目标

的对应) 做家族校正, 不用 Bonferroni。理由是它回答的问题更对得上: 本文要防的是“从 8 个候选里挑最大”这个动作本身带来的偏差, 而置换零假设正是直接模拟这个动作——即使预测变量与目标毫无关联, 纯噪声的 $\max |\rho|$ 在 $n = 10$ 下也有 95% 的机会达到 0.81。(Bonferroni 只把 p 门槛收到 0.00625, 它假定各候选相互独立; 而本文的候选都是同一批前沿的不同摘要、彼此高度相关。两者的口径并不相同, 本文不做换算。)

5.4 复现环境

代码为 Python 实现, 数值依赖 NumPy/SciPy; 实验按实例分批驱动并支持断点续跑。论文的每一个数字都由 `scripts/paper_export.py` 从结果文件现场生成, 取不到即报错, 不允许人工填数。

6 结果

本节分四步给出结果。§6.1 逐级打开六个臂, 看每个组件各自贡献多少; §6.2 用算力响应面(把算力当自变量、看效应如何随它变化)回答“是不是因为算力不够”; §6.3 厘清两个目标量的区别; §6.4 提出并检验零代探针。

6.1 组件阶梯: 只有初始化与随机 VNS 站得住

表 3 与图 3 给出六臂阶梯的逐级增量。在实例边界口径下: MIX3 初始化 +3.406% (10/10 实例胜出, $p = 0.002$)、随机 VNS +0.421% (10/10, $p = 0.002$)、精英档案 +0.082% (10/10, $p = 0.002$); 而 Q-PAS 只有 +0.030% (5/10, $p = 0.625$, 不显著), RL 选算子 -0.003% (6/10, 不显著)。注意最后一行的含义: RMOEAD 与 D_5 的唯一差别是“用 RL 选算子”还是“随机选算子”, 等算力、同其它一切——把 RL 组件换成随机选择, 结果没有区别。

表 3: 组件阶梯的逐级增量: 实例边界口径 (正文引用) 与盒口径 (仅作对照)

阶梯	组件	实例边界口径				盒口径		
		ΔHV 相对增幅 (%)	实例胜出	d_z	p	相对增幅 (%)	放大	
D2 D1	MIX3 初始化	+3.406	10/10	+0.98	$2.0 \times 10^{-3**}$	+19.956	5.9×	
D3 D2	随机 VNS	+0.421	10/10	+0.61	$2.0 \times 10^{-3**}$	+4.199	10.0×	
D4 D3	Q-PAS	+0.030	5/10	+0.05	0.625	+0.314	10.6×	
D5 D4	精英档案	+0.082	10/10	+0.65	$2.0 \times 10^{-3**}$	+0.400	4.9×	
RMOEAD D5	RL 选算子	-0.003	6/10	-0.005	0.625	-0.091	29.7×	

Mk01-Mk10, 每格 $n = 30$ seeds 同 seed 配对。 ΔHV 相对增幅为逐实例 $\Delta\text{HV}/\text{HV}_{\text{上一级}}$ 的跨实例均值 (不用 Δ/HV : 效应 ≈ 0 时会反号)。 d_z 为 run 级池化配对效应量 ($n = 300$), p 为实例级 Wilcoxon ($n = 10$)。盒口径一列是同一批数据换口径重算的结果, 用来量化口径的影响; 正文的效应量一律取实例边界口径。

6.2 算力-算法响应面: 不是学不到, 是没空间可学

图 4 把每个臂对放在同一族“墙钟预算分数”(10%, 25%, 50%, 75%, 100%, G 上限 2000) 上比较。初始化的效应随算力增加而显著衰减: 在最紧的一档预算 (10%) 上, 三个实例的 $|\Delta\text{HV}|$

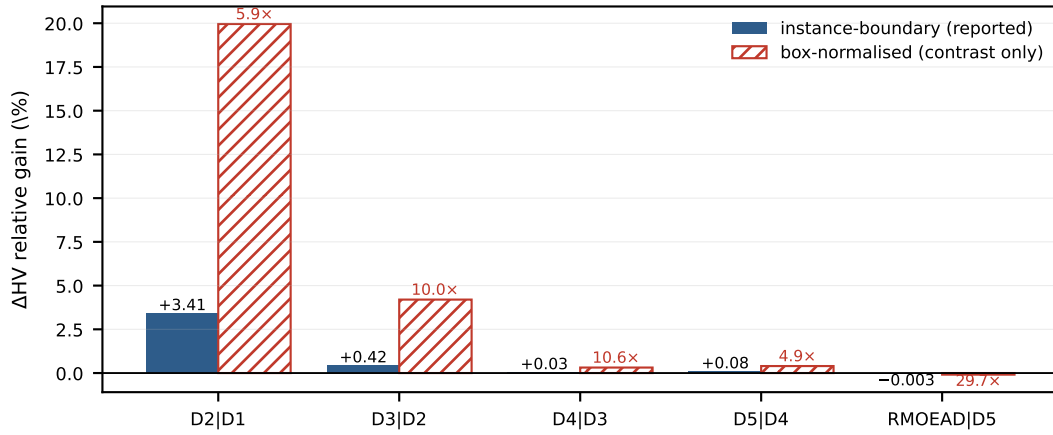


图 3: 六臂阶梯的逐级 ΔHV 相对增幅 (纵轴, 单位%) 与两种口径的对照。横轴每一格是一个“臂对”, $D_2|D_1$ 读作 “ D_2 相对 D_1 多出来的那部分”, 自左向右即 §3 中阶梯逐级打开的顺序。实心柱为实例边界口径 (正文引用), 斜纹柱为盒口径 (仅作对照), 柱顶红色数字是两者的倍数。可以看出盒口径把接近零的效应相对放大得最多。

为 0.024/0.064/0.091, 到全额预算时只剩 0.0010/0.0028/0.0085 (衰减 10.7–23.8 倍); 而 RL 选算子在**所有**预算档上都贴着零线。表 4 给出全额预算下的绝对值: 非初始化组件的最大者也只有 1.1×10^{-3} (Mk09 的 RL 选算子)。

表 4: 算力全部花完时 (100% 预算) 各组件相对上一级的 ΔHV 绝对值

臂对	组件	Mk07	Mk09	Mk10
D2 D1	MIX3 初始化	+0.001	+0.003	+0.009
RMOEAD D5	RL 选算子	+0.0003	-0.001	-0.0002

每列一个实例; 预算按**墙钟** (anytime 轨迹) 分五分位 (10%, 25%, 50%, 75%, 100%, G 上限 2000)。把算力加到上限以后, 除初始化以外的组件增量都不超过 1.1×10^{-3} (最大者为 Mk09 的 RL 选算子, 绝对值)。

6.3 同一批数据上的两个目标量

先说清两种被比较的起点。任何进化算法都要先造一批初始排产方案 (**初始化**), 再从这里开始搜索; 起点好不好对结果影响很大, 这正是 §6.1 里初始化成为最大组件的原因。本文涉及两种:

- **MIX3** (复现对象自带的初始化, 也是“目标 B”的对照): 把初始种群三等分, 各用三条规则之一各造三分之一——完全随机; 每道工序挑 最可能加工时间最短的机器; 每道工序挑当前负载最轻的机器。注意三条规则都只在“选哪台机器”上做文章, **工序的先后顺序仍然是随机打乱的**。
- **MWR** (本文的候选变体): 把工序的先后顺序也管起来——按 剩余工时最多者优先排列 (一种常见的派工启发式, 先把长活排上队), 机器维度则保留随机性。

于是“该不该用 MWR”有两种问法, 对应两个不同的目标量: 拿**论文自己的 MIX3 初始化**做对照 (目标 B), 问的是**净增量**——“在论文已经达到的水平上还能多赚多少”; 拿**随机初始化**做对照 (目标 A), 问的是“有个好起点本身值多少”。两个量在同一批数据上都算得出来, 但用

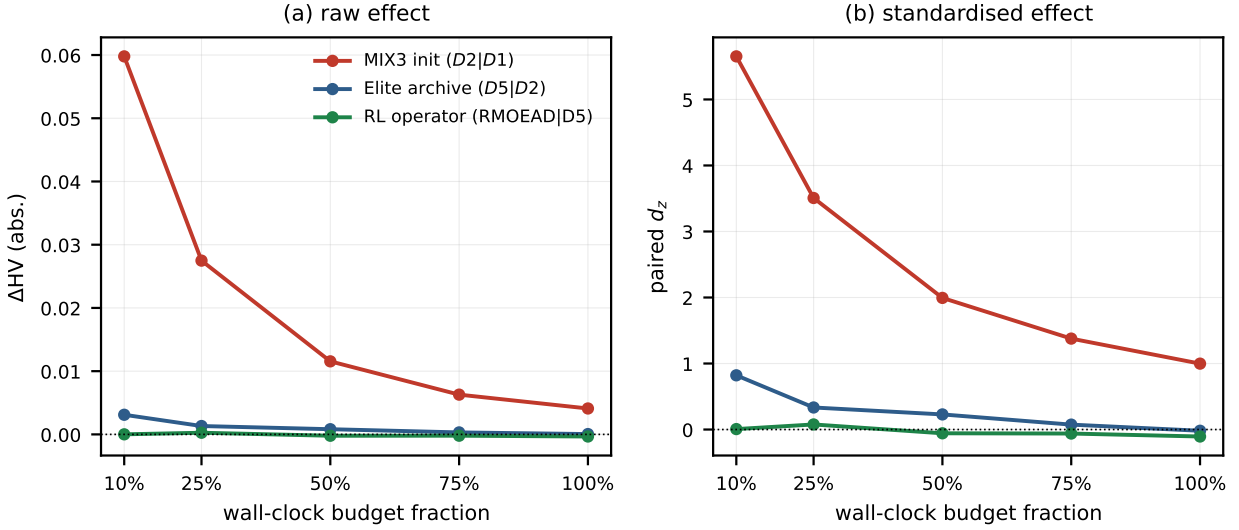


图 4: 算力-算法响应面。两栏的横轴都是墙钟预算分数 (10%, 25%, 50%, 75%, 100%, G 上限 2000), 纵轴分别为 ΔHV 绝对值与配对效应量 d_z ; 每个点是三个实例的均值, 曲线对应一个臂对 (即一个组件)。初始化 (红) 是唯一随算力增加而衰减的显著效应, RL 选算子 (绿) 全程贴零。

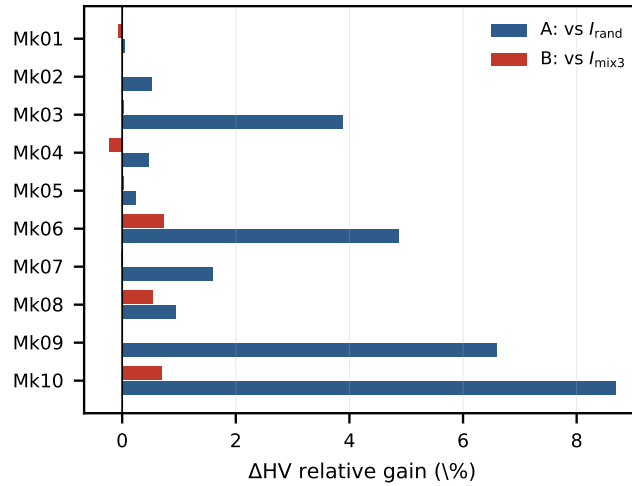


图 5: 两个目标量的逐实例比较。目标 A (蓝) 处处为正; 目标 B (红) 只在 Mk06/Mk08/Mk10 上显著为正, 其余七个与 0 不可区分。

目标 A 的答案去论证目标 B 是无效的: 前者可以很大 (好起点普遍有价值), 后者却可能一点不剩。表 5 与图 5 同时给出两者。

- **目标 A** (I_{mwr} vs. I_{rand}): 10/10 实例为正、8 个显著 (只有 Mk01 与 Mk05 不显著), 幅度 +0.05% 至 +8.69%。这说明 “有个好起点” 普遍有价值。
- **目标 B** (I_{mwr} vs. I_{mix3} , 净增量): 只有 3/10 显著, 幅度 +0.54% 至 +0.73%; 其余 7 个与 0 不可区分 (-0.22% 至 +0.02%, 全部不显著)。

因此 MWR 的净增量是**全有或全无**的: 它在少数实例上有值, 在其余实例上毫无增益。把目标 A 的读数当作目标 B 的答案, 正是本文要提醒的第二类错位; 它会把 “Mk10 上有净增益” 错误地推广为 “该初始化普遍更好”。

表 5: 同一批数据、两个目标量: 目标 A (I_{mwr} vs. I_{rand}) 与目标 B (I_{mwr} vs. I_{mix3})

实例	目标 A: 有好起点值多少				目标 B: MWR 相对论文口径的净增量		
	$\Delta\text{HV}(\%)$	胜出	d_z	p	$\Delta\text{HV}(\%)$	胜出	p
Mk01	+0.047	13/30	+0.06	0.685	-0.073	15/30	0.626
Mk02	+0.525	22/30	+0.74	$5.5 \times 10^{-4***}$	+0.010	17/30	0.556
Mk03	+3.879	30/30	+3.88	$< 10^{-4***}$	+0.023	17/30	0.328
Mk04	+0.467	21/30	+0.52	0.013*	-0.224	12/30	0.171
Mk05	+0.238	16/30	+0.24	0.184	+0.024	15/30	1.000
Mk06	+4.871	30/30	+5.76	$< 10^{-4***}$	+0.730	29/30	$< 10^{-4***}$
Mk07	+1.593	30/30	+3.25	$< 10^{-4***}$	-0.009	16/30	1.000
Mk08	+0.944	29/30	+1.55	$< 10^{-4***}$	+0.542	28/30	$< 10^{-4***}$
Mk09	+6.584	30/30	+5.88	$< 10^{-4***}$	+0.012	18/30	0.685
Mk10	+8.692	30/30	+8.25	$< 10^{-4***}$	+0.690	29/30	$< 10^{-4***}$

$n = 30$ seeds, 同 seed 配对; ΔHV 为实例边界口径的相对增幅。目标 A 上 10/10 为正 (“好起点”这一级确有普遍收益), 但目标 B 只有 3/10 显著、其余 7 个与 0 不可区分 (配对 Wilcoxon $p \geq 0.17$; 注意 ΔHV 并非精确的 0, 非显著实例中仍为正的最大者是 +0.024%, 故 “有增益” 必须按显著性而非非零来判): MWR 的净增量是全有或全无, 并非 “普遍成立但幅度较小”。拿 A 的结论回答 B 的问题即为目标量错位 (缺陷 26)。

6.4 零代探针: 把事后解释升级为事前判据

既然净增益是 “全有或全无”, 一个自然的问题是: 能不能事前知道某个实例属于哪一类? 最直接的判别量是 “初始化的 makespan/HV 杠杆”, 但它通常是事后量 (从末代档案算)。我们提出的探针只调用一次初始种群构造 (等价于 $G = 0$, 一次搜索都不做), 比较两个变体初始前沿 HV 的相对差:

$$\lambda_0 = \frac{\overline{\text{HV}}_0(I_{\text{mwr}}) - \overline{\text{HV}}_0(I_{\text{mix3}})}{\overline{\text{HV}}_0(I_{\text{mix3}})} \times 100\%, \quad (2)$$

其中 $\overline{\text{HV}}_0$ 为 30 个 seed 的初始前沿 HV 均值 (实例边界口径, ref 与正式实验相同)。代价是每实例 $\times 30$ seeds 约 1.4s。

在开发集 (Mk01–Mk10) 上, 净增益为正的 3 个实例的 λ_0 都落在 $[-0.53\%, +0.64\%]$ 内, 其余七个都低于门限 -1.0% (最接近的 Mk09 为 -1.06%); 取门限 $\lambda_0 \geq -1.0\%$ 即 **完全分离**: 3/3 命中、0 误放、0 漏放。该门限在开发集上标定后冻结, 不再按留出集调整。

这里必须把两个相关系数分开报, 否则会重犯本文所批评的那类错位。(下文 ρ 一律指 **Spearman 秩相关系数**: 把 λ_0 与净增量各自排名次, 看两个名次同向的程度——+1 表示完全同向, 0 表示毫无关系。) λ_0 自身 $\rho = +0.77$ (名义 $p = 0.009$); 若改取 9 个事前候选的家族最大, 最高者并不是 λ_0 而是 “初始种群平均 makespan 杠杆”, $\rho = +0.83$ 、置换校正 $p = 0.035$ 。注意家族零假设下 $\max |\rho|$ 的 95% 分位为 0.81, 即 λ_0 自身过不了这道家族校正 ($+0.77 < 0.81$)。因此我们不以 ρ 的显著性作为门控依据, 只以下面的门限分类结果为准; 这也正是必须做独立留出验证的原因。

需要强调的是, 事后量 “最终 makespan 杠杆” 与净增益的相关更高 ($\rho = +0.95$), 但**用它做门控是循环论证**: 要算它必须先把两个臂都跑完。探针的价值恰恰在于代价只有一次初始种群构造。

7 独立留出验证：Hurink e-data 的 66 个实例

为什么还需要一个独立留出集：开发集只有 $n = 10$ ，§6.3 “3 个显著、7 个与 0 不可区分”这个形态本身就建立在这点样本上，而门限又是从同一批实例上挑出来的，因此开发集上的“完全分离”不能作为证据。我们把门限 $\lambda_0 \geq -1.0\%$ 原样搬到留出集——实例是 Hurink e-data 的 **全部 66 个**，不做任何按结果的挑选；每个实例跑 I_{mwr} 与 I_{mix3} 两个臂、每臂 30 个种子，参数、算力与开发集完全一致。

留出集上净增量显著为正的实例为 10/66 (15.2%)，门控命中 3、误放 13、漏放 7，判对率 0.70。但留出集本身只有 15.2% 的实例真赚，**一律拒绝**的判对率就有 0.85——门控比它低 0.152，而门控放行的 16 个实例里只有 3 个真赚（精确率 0.19）。换言之，在这个分布上探针的判别力**低于不做任何判别**。其中 15 个实例的 λ_0 落在门限 ± 0.5 个百分点内——这一带在开发集上已被证明即使 30 seeds 也判不稳 (§8)，故判对率要连同它一起读。

表 6 把两个集合的判定结果并排列出，即**混淆矩阵**——“真赚/真不赚”与“放行/拦下”的四种组合各有多少个实例；图 6 把同一份判定画在 λ_0 轴上，可以直观看出门限两侧的混叠；表 7 则逐实例列出探针 λ_0 与目标 B 的净增量。留出集的 λ_0 取值范围是 $[-5.8\%, +0.6\%]$ ，而开发集正例的 λ_0 落在 $[-0.53\%, +0.64\%]$ ——两个集合的 λ_0 分布并不相同，位置与跨度都对不上，这正是“门限在开发集上冻结、再到留出集检验”要暴露的东西：门控可靠与否，取决于留出集的 λ_0 有多少落在门限的决策边界附近。

这个判对率必须与平凡基线并列看，否则容易读反。留出集上真正有增益的实例只有 10/66，所以**不加判别、一律拒绝**就能得到 0.85 的判对率，**比门控的 0.70 还高 0.152**。门控一共放行了 16 个实例，其中只有 3 个真赚（精确率 0.19）；而 10 个真赚的实例里有 7 个被拦下（召回 0.30）。也就是说，在这个分布上，探针的判别**不如不做判别**。

这个负结果要读准两点。其一，它**不说明 MWR 无效**——净增量是否普遍存在，由 §6.3 的目标 B 回答。它说明的是“**用一个零代探针事前判断该不该用它**”这个方案在跨分布时不可靠：门限是在 $n = 10$ 的开发集上挑出来的（在那里它把全部 10 个实例都判对，判对率 1.00），而 Hurink 上 λ_0 的分布与开发集并不相同， $[-5.8\%, +0.6\%]$ 的跨度远大于开发集正例所在的 $[-0.53\%, +0.64\%]$ 。

其二，**判对率低有一部分确实来自决策边界**：15 个实例的 λ_0 落在门限 ± 0.5 个百分点内，而 §8 已证明这一带即使 30 个 seed 也判不稳。但**精确率与召回不受这个解释的庇护**：即使把所有贴边实例都判对，门控放行的实例里也仍只有 0.19 真赚。所以跨分布的可移植性不是靠调门限能补的。

表 6: 零代门控的混淆矩阵（门限在开发集上标定后冻结： $\lambda_0 \geq -1.0\%$ ）

集合	n	命中	误放	漏放	正确拒绝	判对率
开发集 Mk01-Mk10	10	3	0	0	7	1.00
留出集 Hurink e-data	66	3	13	7	43	0.70

“有增益”按配对 Wilcoxon $p < 0.05$ 且 $\Delta\text{HV} > 0$ 判定。“命中”= 预测有增益且确实有；“误放”= 预测有而实际与 0 不可区分；“漏放”= 预测没有而实际有。开发集是门限的标定集，只有留出集一列是无偏的。

表 7: 独立留出集 (Hurink *e*-data, $n = 66$): 零代探针 λ_0 与 MWR 净增量 (左右两栏续排: 左栏 Hed01–Hed33, 右栏 Hed34–Hed66)

实例	$\lambda_0(\%)$	$\Delta HV(\%)$	胜出	d_z	p	显著	实例	$\lambda_0(\%)$	$\Delta HV(\%)$	胜出	d_z	p	显著
Hed01	-4.31	-0.106	9/30	-0.12	0.445	–	Hed34	-3.96	+0.308	15/30	+0.16	0.746	–
Hed02	+0.56	+0.298	18/30	+0.36	0.080	–	Hed35	-2.95	+0.778	20/30	+0.34	$9.9 \times 10^{-3**}$	✓
Hed03	-2.03	-0.006	17/30	-0.005	0.968	–	Hed36	-2.93	+0.290	14/30	+0.16	0.465	–
Hed04	-3.98	+0.153	21/30	+0.33	0.025*	✓	Hed37	-2.35	+0.378	18/30	+0.17	0.198	–
Hed05	-5.79	+0.131	17/30	+0.09	0.482	–	Hed38	-0.71	+0.400	18/30	+0.14	0.213	–
Hed06	-2.12	-0.087	14/30	-0.10	0.465	–	Hed39	-1.95	-0.146	14/30	-0.09	0.584	–
Hed07	-1.87	+0.160	15/30	+0.22	0.416	–	Hed40	-0.63	+0.497	21/30	+0.40	0.028*	✓
Hed08	-5.01	+0.243	15/30	+0.34	0.265	–	Hed41	-1.10	+0.808	24/30	+0.60	$1.9 \times 10^{-3**}$	✓
Hed09	-4.03	+0.015	15/30	+0.04	0.888	–	Hed42	-1.12	+0.086	16/30	+0.07	0.919	–
Hed10	-4.97	-0.085	13/30	-0.14	0.393	–	Hed43	-1.53	+0.170	18/30	+0.18	0.428	–
Hed11	-1.06	+0.058	14/30	+0.15	0.777	–	Hed44	-1.16	+0.446	25/30	+0.90	$< 10^{-4***}$	✓
Hed12	-3.67	+0.015	15/30	+0.05	0.442	–	Hed45	-0.33	+0.056	16/30	+0.10	0.570	–
Hed13	-2.86	+0.029	14/30	+0.05	0.626	–	Hed46	-1.54	-0.237	17/30	-0.15	1.000	–
Hed14	-2.74	-0.001	14/30	-0.003	0.919	–	Hed47	-0.02	+0.369	20/30	+0.23	0.146	–
Hed15	-2.45	+0.065	15/30	+0.18	0.516	–	Hed48	-0.70	-0.312	12/30	-0.20	0.164	–
Hed16	-5.19	-0.197	9/30	-0.43	0.031*	–	Hed49	-0.04	+0.428	17/30	+0.18	0.371	–
Hed17	-2.48	+0.136	18/30	+0.34	0.140	–	Hed50	-3.52	-0.330	15/30	-0.20	0.584	–
Hed18	-0.78	+0.111	17/30	+0.15	0.371	–	Hed51	-2.33	+0.241	17/30	+0.12	0.529	–
Hed19	-4.82	-0.130	12/30	-0.18	0.416	–	Hed52	-0.93	+0.092	12/30	+0.06	0.952	–
Hed20	-2.84	+0.030	15/30	+0.04	0.855	–	Hed53	-1.57	-0.013	16/30	-0.009	1.000	–
Hed21	-4.61	+0.016	19/30	+0.02	0.808	–	Hed54	-2.28	-0.377	14/30	-0.21	0.529	–
Hed22	-3.74	+0.530	20/30	+0.55	$8.7 \times 10^{-3**}$	✓	Hed55	+0.20	+0.730	19/30	+0.34	0.064	–
Hed23	-0.32	+0.227	19/30	+0.22	0.146	–	Hed56	-0.22	-0.177	12/30	-0.10	0.404	–
Hed24	-0.89	-0.052	17/30	-0.05	0.871	–	Hed57	-2.83	+0.050	17/30	+0.04	0.777	–
Hed25	-3.49	+0.166	18/30	+0.18	0.114	–	Hed58	-0.68	+0.321	21/30	+0.42	0.028*	✓
Hed26	-1.04	+0.193	21/30	+0.28	0.050*	✓	Hed59	-1.26	+0.091	17/30	+0.08	0.700	–
Hed27	-4.37	-0.013	17/30	-0.02	0.700	–	Hed60	-0.59	+0.452	20/30	+0.61	$3.0 \times 10^{-3**}$	✓
Hed28	-3.12	+0.340	20/30	+0.31	0.070	–	Hed61	-3.26	-0.116	14/30	-0.08	0.626	–
Hed29	-1.93	+0.157	20/30	+0.12	0.360	–	Hed62	-2.05	+0.068	17/30	+0.06	0.715	–
Hed30	-1.53	+0.387	19/30	+0.26	0.105	–	Hed63	-2.65	+0.252	18/30	+0.35	0.100	–
Hed31	-3.35	-0.218	15/30	-0.21	0.416	–	Hed64	-2.87	-0.066	15/30	-0.06	0.871	–
Hed32	-1.92	+0.429	21/30	+0.35	0.040*	✓	Hed65	-1.62	-0.223	14/30	-0.24	0.191	–
Hed33	-3.30	+0.060	19/30	+0.06	0.839	–	Hed66	-0.83	+0.212	17/30	+0.16	0.477	–

λ_0 为零代探针给出的初始前沿 HV 杠杆 (MWR – MIX3, 只调一次 `_init_population()`, 不做任何搜索)。门限 $\lambda_0 \geq -1.0\%$ 在开发集 (Mk01–Mk10) 上标定后冻结。「显著为正」= $\Delta HV > 0$ 且配对 Wilcoxon $p < 0.05$ ——不是「 $\Delta HV \neq 0$ 」: 两个实例集上都没有 ΔHV 精确为 0 的实例 (开发集最小非零为 -0.009%), 用非零当判据会把全部实例判成有增益。留出集上显著为正的实例为 10/66, 门控判对 46/66。

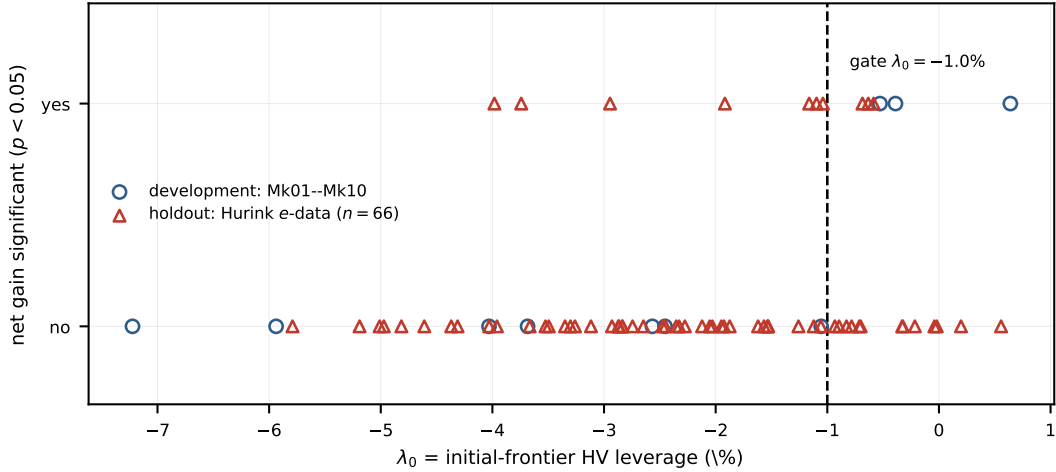


图 6: 零代门控的分离情况: 横轴为探针读数 λ_0 (初始前沿 HV 杠杆, %), 纵轴为“该实例的净增量是否显著为正”(是 = 上排, 否 = 下排), 每个标记是一个实例。虚线即门限 $\lambda_0 = -1.0\%$, 它在开发集 (圆点) 上标定后立即冻结, 再原样用到留出集 (三角) 上。门控理想的效果是“门限右侧全在上排、左侧全在下排”, 实际并非如此: 留出集在门限右侧仍有许多三角落在下排——被放行的实例里有相当一部分其实并不赚。

8 门控需要几个 seed ?

零代探针虽然便宜, 但它仍需要 30 个 seed。开发集上两类实例的 λ_0 只相隔约 0.5 个百分点 (正例最低 -0.53% , 非正例最高 -1.06%), 因此“少采几个 seed 会不会翻掉判定”必须实测, 否则“廉价门控”的说法是不成立的。

做法是: 对每个实例先算出 30 个 seed 的逐 seed 值, 再对 $n \in \{1, 2, 3, 4\}$ 枚举全部 $\binom{30}{n}$ 个子集、对更大的 n 随机抽样 2000 个子集, 用与正式探针完全相同的公式重算 λ_0 , 最后看这样得到的判定是否与该实例用全部 30 个 seed 得到的判定一致。表 8 给出结果。

结论是分层的: 离门限远的实例在 $n = 1$ 就已经稳定, 而贴着门限的实例 (如 Mk09, $\lambda_0 = -1.06\%$, 只比门限低 0.06 个百分点) 即使到 $n = 10$ 也只有 0.73 的一致率。跨实例平均一致率在 $n = 1$ 时约 0.95, 看起来很好, 但最差实例只有 0.50 ——这个平均值把两类实例混在一起, 掩盖了边界实例的不稳定。因此正确的用法是: 除了门限判定, 还要看到门限的距离; 距离小于约 0.62 个百分点的实例应被标为“不确定”, 或加大 seed 预算后再判。

9 讨论

9.1 为什么两个 RL 组件没有空间

三条证据互相独立且一致: (1) 阶梯上 Q-PAS 与 RL 选算子均不显著; (2) 把 T 的动作空间 (强化学习术语, 指算法可做的选择集合, 这里即“ T 的全部候选值”) 穷举扫描一遍, 其中也包含动作空间之外的更大候选值: 空间内最优与最差的差只有 $+0.128\%$ (配对检验 $p = 0.073$ 、效应量 $d = +0.37$, 胜出 19/30), 空间内那个最优的 T 同时就是全部 6 档候选中的最优——动作空间没有漏掉任何有意义的东西, 故动作空间本身的天花板就很低; (3) 算力响应面显示 $G = 200$ 附近已接近饱和, 把预算放大到 $G = 2000$ (即 $10 \times$ 算力) 也只带来 $+1.43\%$ 的相对增幅。把三者

表 8: 零代探针在不同 seed 预算下的稳定性 (Mk01–Mk10)

实例	$\lambda_0(30)$	λ_0 的子集标准差				与 30-seed 判定的一致率			
		$n=1$	$n=3$	$n=5$	$n=10$	$n=1$	$n=3$	$n=5$	$n=10$
Mk01	−7.22	1.34	0.74	0.56	0.35	1.00	1.00	1.00	1.00
Mk02	−2.45	0.63	0.35	0.26	0.16	1.00	1.00	1.00	1.00
Mk03	−4.03	0.27	0.15	0.11	0.07	1.00	1.00	1.00	1.00
Mk04	−5.94	1.19	0.65	0.48	0.31	1.00	1.00	1.00	1.00
Mk05	−3.68	0.85	0.47	0.35	0.22	1.00	1.00	1.00	1.00
Mk06	−0.39	0.33	0.18	0.13	0.08	1.00	1.00	1.00	1.00
Mk07	−2.57	0.42	0.23	0.17	0.11	1.00	1.00	1.00	1.00
Mk08	+0.64	0.48	0.26	0.20	0.13	1.00	1.00	1.00	1.00
Mk09	−1.06	0.36	0.19	0.14	0.09	0.50	0.60	0.65	0.73
Mk10	−0.53	0.32	0.17	0.13	0.08	0.97	1.00	1.00	1.00
跨实例均值	−	0.618	0.339	0.254	0.159	0.947	0.960	0.965	0.973

对每个实例枚举/抽样 n 个 seed 的子集，用与正式探针完全相同的估计量（先对 seed 求均值再作比）重算 λ_0 ，再看判定是否与 30-seed 一致。门限 −1.0%；子集数见 `logs/probe_seed_var.json`。

放在一起，RL 组件失效的解释不是“学习算法不行”，而是**它在两个方向上都无事可做**：可选的 T 值彼此差别不大，而多算一点也没有回报。反之，初始化改变的是搜索的**起点**，它的效应不随算力增长而被稀释（图 4 的衰减正是“起点优势被后续搜索逐步追平”），因此是这里唯一稳健的杠杆。

9.2 口径问题在复现研究中的普遍性

“取参与比较的前沿极值”作为归一化边界，在文献与开源实现里都很常见，它的问题不是“不精确”而是**不可比**：读数依赖于同批实验的设计。本文的量化结果是，同一批数据下效应量被放大 $4.9\times$ 至 $29.7\times$ （逐格子最高 $47.9\times$ ，由 `caliber_audit.py` 与 `paper_export.py` 两条独立路径算出同一组数），且倍数不是常数；显著性判定与胜出场数也会变。更值得警惕的是它改变现象的形状——把“少数实例显著、其余与 0 不可区分”抹成“处处微正”，后者恰好支持“效应弱但普遍”这个错误结论。我们建议复现研究把口径写成一条**可执行的纪律**（本文 §4），并在论文里同时给出对照，让读者能自行折算。

9.3 对“组件有效性”这类结论的建议

本文的负面结论并不是“这个算法不好”，而是指出**在这批基准与这个算力预算下**，两个学习组件没有可实现空间。要让“某学习组件有效”成为可靠主张，至少需要：与要回答的问题对齐的对照（目标 B 而非 A）、等算力的比较、效应量与显著性两套读数、以及**在未参与任何调参的实例上复现**。本文的 Mk06 与 Mk08 属于后一类：它们从未参与初始化变体的调参，却复现了 Mk10 上的净增益方向，这比“在调参实例上更好”可信得多。

9.4 局限

- 留出集只有 Hurink *e*-data 一个族，而**它已经把这个门控否掉了 (§7)**。所以这里剩下的未知并不是“会不会失效”，而是换一个族（如 *r/v*-data）是否同样失效——按本文证据，使用时应默认**跨族不可移植**。
- 门限 $\lambda_0 \geq -1.0\%$ 是在 $n = 10$ 的开发集上挑出的，且**没有做跨族校准**——§7 表明这正是它失效的原因之一。要让这类门控可用，**必须在目标分布上重新标定**（本文不这么做，因为那等于把留出集变成标定集）；而 §8 给出的边界不稳定性说明，即便重标定，也必须连同“距门限的距离”一并报告。
- 本文只报告 HV 单一指标；虽然 HV 是双目标问题的常用汇总，但不排除某些组件在特定目标方向上有局部作用。
- 响应面实验受墙钟预算上限限制 ($G \leq 2000$)，“饱和”的结论限定在该范围内。

10 结论

我们在统一、可复现的评价口径下重新审计了 RMOEA/D 的各组件。主要结论是：在本基准与算力预算下，真正有效的是**初始化与算力总量**（且后者很快饱和），两个 RL 组件（Q-PAS 选 *T*、RVNS 选算子）没有可实现空间。同时我们发现并量化了一个被普遍忽视的评价口径问题——归一化边界的取法会把效应量放大一个数量级、改变显著性判定、甚至改变现象的形状。

在建设性的一面，我们给出一个零代探针：它以一次初始种群构造的代价，事前判定“某新实例上初始化的净增益是否会出现”，并在 66 个独立实例上做了预注册的留出验证。**该门控没有通过**：留出集上的判对率 0.70 低于“一律拒绝”这一平凡基线 0.85，放行实例的精确率仅 0.19，召回仅 0.30。因此这一条以**负结果**呈现：**廉价的事前门控在这批基准上不可移植**。这类门控在文献中常被视为“顺手可得”的组件，因此值得被显式证伪。

我们希望本文的方法学部分——口径纪律、目标量对齐、置换零假设、以及“把结论放到未调参实例上”——对同类复现工作有直接的可操作性。

参考文献

- [1] J. Li, et al. A reinforcement learning based RMOEA/D for bi-objective fuzzy flexible job shop scheduling. *Expert Systems with Applications*, 203:117380, 2022.
- [2] K. Deb, et al. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE TEC*, 6(2):182–197, 2002.
- [3] Q. Zhang, H. Li. MOEA/D: A multiobjective evolutionary algorithm based on decomposition. *IEEE TEC*, 11(6):712–731, 2007.
- [4] P. Brandimarte. Routing and scheduling in a flexible job shop by tabu search. *Annals of Operations Research*, 41(3):157–183, 1993.
- [5] E. Hurink, B. Jurisch, M. Thole. Tabu search for the job-shop scheduling problem with multi-purpose machines. *OR Spektrum*, 15(4):205–215, 1994.